

PRUEBA TÉCNICA

Introducción

Esta prueba evalúa tu capacidad para construir un pipeline de Machine Learning completo orientado a recomendación de productos, similar al núcleo técnico de plataformas de e-commerce.

El reto está dividido en tres etapas progresivas: ranking supervisado, representación vectorial de productos y orquestación mediante un agente conversacional.

Dataset

<https://www.kaggle.com/datasets/yasserh/instacart-online-grocery-basket-analysis-dataset>

Archivo	Descripción
orders.csv	Historial de órdenes por usuario (cuándo, qué día, qué hora)
order_products__prior.csv	Productos comprados en órdenes anteriores
order_products__train.csv	Última orden de cada usuario (ground truth)
products.csv	Catálogo de productos con nombre
aisles.csv	Pasillos / subcategorías
departments.csv	Departamentos / categorías principales

Etapla 1 — Learning to Rank (30 pts)

Objetivo: Dado un usuario y su historial, rankear una lista de productos candidatos de mayor a menor probabilidad de ser comprados en su próxima orden.

Contexto del problema

En una plataforma como e-commerce, predecir si un producto se compra o no es insuficiente — necesitas ordenar miles de productos y mostrar los más relevantes primero. Eso es un problema de ranking, no de clasificación binaria.

Construcción del dataset

Usando `order_products__prior.csv`, `orders.csv` y `products.csv`, el candidato debe construir pares (usuario, producto) con un label de relevancia:

Label	Significado
2	Producto comprado Y reordenado (reordered = 1)
1	Producto comprado por primera vez en esa orden
0	Producto candidato que NO fue comprado (negative sampling)

Para los negativos, samplear productos del mismo departamento que el usuario nunca ha comprado. Justificar cuántos negativos por positivo se usaron y por qué.

Features requeridas

- Por cada par (usuario, producto) construir las siguientes 4 features:
- `user_product_frequency` — número de veces que el usuario compró ese producto en su historial
- `user_product_reordered` — si el usuario alguna vez reordenó ese producto (1 / 0)
- `product_global_popularity` — número total de usuarios distintos que han comprado ese producto
- `department_id` — departamento del producto (label encoding)

Modelo

Usar cualquier rankers conocido o uno de los siguientes:

- XGBRanker con `objective rank:ndcg`
- LightGBM con `objective = lambdarank`

Requisito crítico: usar correctamente el parámetro `group` (o `query`) del ranker, que agrupa los candidatos por usuario/orden. Sin esto el modelo no aprende a rankear correctamente.

Evaluación del modelo

- Split: usar `order_products__train.csv` como set de test (última orden real de cada usuario)
- Métricas requeridas: `NDCG@10` y `MAP@10`
- Baseline obligatorio: rankear por popularidad global (`product_global_popularity` descendente) y comparar contra el modelo entrenado. Si el modelo no supera el baseline, explicar por qué.

Lo que se evaluará

- Construcción correcta del dataset con negative sampling justificado
- Uso correcto del parámetro `group` en el ranker
- Que las métricas `NDCG@10` y `MAP@10` estén bien calculadas

Etapla 2 — Embeddings y similitud semántica (30 pts)

Objetivo: Dado un producto, encontrar los **K productos más similares** usando representaciones vectoriales.

Enfoque esperado:

Entrenar embeddings de productos basados en co-ocurrencia en órdenes (productos que se compran juntos tienden a ser similares). Se puede usar:

- `sentence-transformers` sobre el nombre del producto

Implementar:

- Función `get_similar_products(product_id, k=10)` que retorne los K más similares con su score de similitud coseno
- Visualización opcional con UMAP o t-SNE de los embeddings

Se evaluará:

- Calidad intuitiva de los resultados (¿tiene sentido que "leche" sea similar a "yogurt"?)
- Eficiencia: no debe tardar más de 2 segundos por consulta
- Crear una función que mediante una palabra nos ayude con los k vecinos cercanos.

Etapas 3 — Agente conversacional (40 pts)

Objetivo: Construir un agente simple que, dado un mensaje en lenguaje natural, orqueste las etapas anteriores y devuelva una recomendación al usuario.

Ejemplo de interacción:

None

Usuario: "Quiero armar una cesta saludable para el desayuno"

Agente: "Basado en tu historial y productos similares, te recomiendo:

1. Oat Milk (score: 0.94)
2. Greek Yogurt (score: 0.91)
3. Granola Bar (score: 0.88)"

Stack permitido (elegir uno):

- Google ADK (`google-adk`)
- OpenAI Agents SDK
- LangChain + cualquier LLM

Tools/funciones que el agente debe tener:

- `get_user_history(user_id)` → retorna productos más comprados por el usuario
- `get_similar_products(product_id, k)` → llama al modelo de embeddings de la Etapa 2
- `predict_reorder(user_id, product_id)` → llama al clasificador de la Etapa 1

Se evaluará:

- Diseño claro del agente (system prompt, definición de tools)
- Que el agente encadene correctamente las 3 tools
- Manejo de casos edge (usuario sin historial, producto no encontrado)
- Coherencia de la respuesta final al usuario

Entregables

- Repositorio GitHub.
- README.md con instrucciones de instalación y descripción de cada decisión de diseño.
- Notebook o script de evaluación con métricas por etapa.
- Demo funcional: CLI, Streamlit o FastAPI endpoint.

Rúbrica de evaluación

Criterio	Puntaje máximo
Etapas 1 y 2 — Learning to Rank	30 pts
Dataset construido correctamente (labels + negative sampling justificado)	10 pts
Uso correcto del parámetro group en el ranker	8 pts
NDCG@10 y MAP@10 calculados correctamente + comparación vs baseline	12 pts

Etapas 2 — Embeddings**30 pts**Embeddings generados y función `get_similar_products` funcional

12 pts

Función `search_by_keyword` funcional y resultados coherentes

10 pts

Latencia bajo 2 segundos por consulta

8 pts

Etapas 3 — Agente**40 pts**

Diseño del agente (system prompt + definición de tools)

10 pts

Encadenamiento correcto de las 3 tools

15 pts

Manejo de casos edge

10 pts

Coherencia y calidad de la respuesta final al usuario

5 pts

TOTAL**100 pts**